

Amanda Andrade Abreu
Guilherme Amaral Giffoni

Material de Apoio – Seminário de Prolog

PONTIFÍCIA UNIVERSIDADE CATÓLICA DE MINAS GERAIS
Instituto de Ciências Exatas e Informática

Professor: Dr. Marco Rodrigo Costa

Belo Horizonte
2026

Sumário

1	INTRODUÇÃO	3
1.1	Philippe Roussel	3
1.2	Alain Colmerauer	3
2	HISTÓRICO	5
2.1	Antecedentes pré-1972	5
2.2	Nascimento do Prolog	5
3	PARADIGMAS DA LINGUAGEM	7
4	CARACTERÍSTICAS MARCANTES	9
4.1	Comentários	9
4.2	Fatos	10
4.3	Perguntas	11
4.4	Variáveis	11
4.5	Regras	12
4.6	Unificação	14
4.7	Backtracking	17
5	LINGUAGENS RELACIONADAS	19
6	EXEMPLO DE PROGRAMAS EM PROLOG	21
7	CONSIDERAÇÕES FINAIS	23
	REFERÊNCIAS	25

1 Introdução

Prolog foi inventada em Marseilles (na França), em 1972. Philippe Roussel escolheu o nome como uma abreviação para “**programação lógica**”¹, por sugestão de sua esposa no outono de 1972. Pode-se dizer que Prolog foi a prole de um casamento bem-sucedido entre processamento de linguagem natural e prova de teorema automatizada. A ideia de usar uma linguagem natural como o francês para raciocinar e comunicar diretamente com um computador parecia uma ideia *maluca*, apesar disso, foi a base para o projeto definido por Alain Colmerauer no verão da década de 1970. (COLMERAUER; ROUSSEL, 1993)

Notavelmente, Prolog foi peça central do IBM Watson, que em novembro de 2013, venceu os melhores da categoria no jogo televisivo *Jeopardy!*, sem conexão com a internet, e precede em 4 anos os modelos de IA baseados em *transformers* (aprendizado profundo). (RESEARCH, 2013)

1.1 Philippe Roussel

Philippe Roussel

1.2 Alain Colmerauer

Alain Colmerauer, foi um Doutor em ciência da computação, professor na Universidade de Aix-Marselha, e co-criador do Prolog, nascido em 24 de janeiro de 1941 em Carcassonne. Em 2000, a sua empresa de software se especializava em análise e gerência de riscos financeiros e hipoteca. E em 2005, foi comprada pela Experiean, que se tornou parte do negócio de suporte a decisões empresariais, Experian-Scorex. (FISHER, 2017)

Prêmios recebidos:

- Pomme d’Or du Logiciel Francais (1982)
- Michel Monpetit Award (1986)
- Chevalier de la Legion d’Honneur (1986)
- ACP Research Excellence Award (1991)

¹ em francês: “**programmation en logique**”.

2 Histórico

2.1 Antecedentes pré-1972

Prolog é descendente de três principais linhas de pesquisa, sendo elas: PAT (prova automática de teoremas), processamento de linguagem natural e IA. (KÖRNER et al., 2022)

O campo de pesquisa da inteligência artificial nasce por volta de 1956, e deu impulso para que fossem criadas linguagens de programação funcionais, como o LISP, criado em 1958 por McCarthy.

O PAT teve notória presença em um trabalho seminal publicado por Alan Robinson em 1965 onde descrevia lógica computacional baseada no princípio da resolução. O que é o coração dos procedimentos de inferência adotados à programação de lógica.

Enquanto isso, Alain Colmerauer tentava automatizar a conversação homem-máquina, o que o levou a desenvolver Q-systems que serviu para traduzir de inglês para francês os relatórios meteorológicos canadenses. Tudo isso poderia ser escrito em lógica, e inspirado nele, dentre outras coisas, os levou a criar Prolog.

2.2 Nascimento do Prolog

3 Paradigmas da Linguagem

Prolog é em primeiro lugar pertencente à programação lógica, definimos alguns *fatos* da linguagem, suas *regras* e a partir da base de dados, podemos fazer consultas, ficando a cargo do programador apenas descrever as relações.

Depois de lógica, é declarativa ao invés de procedural; ou seja, especificamos apenas os resultados desejados, ao invés de procedimentos detalhados para chegar nele.

Descrevemos alguns fatos e regras e perguntamos aos programas que tentam responder a elas consultando e/ou inferindo o que se sabe a partir desses fatos e regras.

4 Características Marcantes

Na experiência do autor do livro (citar livro), Prolog é uma linguagem que programadores iniciantes acham ser mais compreensiva que programas equivalentes em linguagens “convencionais”. Em contra-partida, programadores experientes nessas mesmas linguagens, apesar de estarem mais bem preparados para os conceitos abstratos que sondam as LPs, podem achar Prolog uma linguagem difícil de se acostumar, e precisam de muito convencimento até considerarem a linguagem como uma ferramenta de programação útil.

Desde seus começos ao redor de 1970, Prolog foi escolhido por muitos programadores para aplicações de computação simbólica, como:

- Lógica matemática
- Resolução de problemas abstratos
- Entender linguagem natural
- banco de dados relacional
- Implementar automação
- Resolução de equações simbólicas
- Análise de estrutura bioquímica
- E várias áreas de inteligência artificial

Prolog permite que um computador seja usado como um armazem de fatos e regras, nos permite a fazer inferências de um fato para outro, encontrar os valores das variáveis pode nos levar a uma dedução lógica.

A sintaxe dos seus programas é muito diferente das linguagens funcionais e imperativas. A semântica também tem pouca semelhança com a dos programas em linguagens imperativas.

4.1 Comentários

```
% esse é um comentário de uma linha só (lembra LaTeX)
mãe(roberta, joaquina) % roberta é mãe de joaquina

/*
rosas são vermelhas
violetas são azuis
```

```
comentário multilinha
adoro cuzcuz
*/
```

4.2 Fatos

Para expressar a relação: “João gosta de Maria”, escrevemos em Prolog:

```
gosta(joão, maria).
```

A ordem dos fatores é arbitrária, mas deve respeitar alguma ordem determinada pelo programador, para manter consistência. No exemplo, levemos em consideração que a pessoa que gosta, seja o primeiro “elemento” do que está entre parênteses. Que não é a mesma coisa que:

```
gosta(maria, joão).
```

Para demonstrar com mais clareza ainda como a consistência é importante na linguagem, observe a seguinte definição de um *fato*:

```
pai(tom, william).
```

Podemos assumir que *tom* é pai de *william*, mas também podemos interpretar que os dois possuem o mesmo pai. A linguagem não restringe a semântica, que por sua vez, é responsabilidade do programador.

Outros exemplos, com as possíveis interpretações:

```
valioso(diamante).      % Diamante é valioso.
mulher(jane).            % Jane é uma mulher.
possui(jane, diamante).  % Jane possui diamantes
pai(alberto, maria).     % Alberto é pai de Maria
da(alberto, livro, maria). % Alberto deu um livro para Maria
```

Note que cada fato descrito acima, termina com um ponto final.

Vamos agora dar as terminologias corretas para essa construção; os *objetos* entre parênteses (de vermelho) são os argumentos, e o nome da relação que vem antes (de azul), é chamado de *predicado*.

O coletivo de fatos é chamado de *base de dados*.

4.3 Perguntas

Com alguns fatos em mãos, podemos fazer algumas perguntas agora. Perguntas são como fatos na perspectiva sintática, exceto que colocamos o símbolo de interrogação seguido de um hífen. Considere:

```
?- proprietario(maria, veiculo).
```

Quando uma pergunta é feita, o sistema vasculhará a base de dados em busca de fatos que unem o fato da questão. Dois fatos *unem* se seus predicados conferem. Se o Prolog encontrar o fato que une com a questão, o mesmo responderá: sim. Se não existir, Prolog responderá não.

Considere a base de dados abaixo:

```
gosta(mario, cogumelo).
gosta(mario, peach).
gosta(peach, toad).
gosta(luigi, toad).
gosta(luigi, moeda).
```

Com esta base de dados, podemos fazer perguntas e o Prolog responderá:

```
?- gosta(mario, moeda).
no
?- gosta(mario, toad).
no
?- gosta(mario, peach).
yes
```

Em prolog, a resposta “no” é usada pra dizer “nada se une com a pergunta”; o que não é a mesma coisa que falso. Apesar de luigi também gostar de cogumelos, não podemos provar isso a partir dos fatos da base de dados (hipótese do mundo fechado).

4.4 Variáveis

As variáveis em Prolog são diferentes da noção convencional, no seu contexto, ela serve para que o sistema responda à pergunta, por exemplo: "do que o mario gosta?", como destacado abaixo:

```
?- gosta(mario, AlgoQueMarioGosta).
```

Note que o segundo argumento começa com uma letra maiúscula! Isto é proposital e é a sintaxe utilizada para marcar variáveis. Quando fazemos uma pergunta ao Prolog em que um dos argumentos é uma variável, a linguagem pesquisa por todos os seus fatos para encontrar um objeto que a variável poderia ser, em termos técnicos: o que pode ser unificado.

Poderíamos reduzir `AlgoQueMarioGosta` por `X` e não teríamos problemas semânticos

```
?- gosta(mario, X).
X = cogumelo
```

A partir desse momento a variável `X` está *instanciada* para `cogumelo`. Se estivermos satisfeitos com a resposta do programa, podemos pressionar **Enter** para terminar a busca neste ponto, caso contrário pressionamos `;`, quando não há mais nada a unificar, a busca termina.

```
?- gosta(mario, X).
X = cogumelo ;
X = peach.
```

Da mesma forma, podemos expressar:

```
?- gosta(X, cogumelo).
X = peach ;
X = luigi.
```

4.5 Regras

E se quiséssemos representar que a `Peach` é a princesa de todos os `Toads`? Uma das formas que podemos fazer isso é escrevendo que `Peach` reina sob cada um dos `Toads`. Por exemplo:

```
princesa(peach, toadette).
princesa(peach, toad).
princesa(peach, toadsworth).
princesa(peach, capitaoToad).
princesa(peach, toadAzul).
% por aí vai...
```

A boa notícia é que há uma melhor forma de descrever essa relação, podemos declarar que a *Peach* é princesa de qualquer objeto que é classificado como um *toad* (como espécie). Este fato, é dado como uma *regra*.

X é um mamífero se:

X possui glândula sudoríparas, e

X possui pelos

ou

X é irmão de Y se:

X é masculino, e

X e Y têm os mesmos pais

Em Prolog, uma regra consiste de um cabeçalho e um corpo. Que são conectados pelo símbolo `:-`, pronunciado como “se”. No exemplo anterior da princesa, podemos expressar essa regra da seguinte forma:

```
% Toadette é da espécie toad
```

```
especie(toadette, toad).
```

```
especie(toad, toad).
```

```
especie(toadsworth, toad).
```

```
especie(capitaoToad, toad).
```

```
especie(toadAzul, toad).
```

```
% Peach é princesa de qualquer um que é da espécie "toad"
```

```
princesa(peach, X) :- especie(X, toad).
```

```
% lê-se como: peach é princesa de X se X é da espécie toad
```

```
?- princesa(peach, X).
```

```
X = toadette ;
```

```
X = toad ;
```

```
X = toadsworth ;
```

```
X = capitaoToad ;
```

```
X = toadAzul.
```

4.6 Unificação

E se quisermos, a partir de predicatos afirmar condições para que algo seja verdadeiro? Ou em português brasileiro, como podemos dizer, por exemplo, que algo é uma linha horizontal ou vertical? Podemos pensar nisso, primeiro de forma imperativa, com C:

```
#include <stdio.h>
#include <stdbool.h>

typedef struct {
    int x;
    int y;
} Ponto;

typedef struct {
    Ponto p1;
    Ponto p2;
} Linha;

Ponto ponto(int x, int y) {
    Ponto p = {x, y};
    return p;
}

Linha linha(Ponto p1, Ponto p2) {
    Linha l = {p1, p2};
    return l;
}

// vertical(linha(ponto(X,Y), ponto(X,Z))) -> mesmo X
bool vertical(Linha l) {
    return l.p1.x == l.p2.x;
}

// horizontal(linha(ponto(X,Y), ponto(Z,Y))) -> mesmo Y
bool horizontal(Linha l) {
    return l.p1.y == l.p2.y;
}

const char* bool_str(bool b) {
    return b ? "true" : "false";
}
```

```

}

int main(void) {
    int x1, y1, x2, y2;

    printf("Digite as coordenadas no formato: x1 y1 x2 y2\n");
    printf("Exemplo: 1 1 1 3\n");

    while (scanf("%d %d %d %d", &x1, &y1, &x2, &y2) == 4) {
        Linha l = linha(ponto(x1, y1), ponto(x2, y2));

        printf("Linha: ponto(%d,%d) -> ponto(%d,%d)\n", x1, y1, x2, y2);
        printf("vertical    ?- %s\n", bool_str(vertical(l)));
        printf("horizontal ?- %s\n", bool_str(horizontal(l)));
        printf("\nDigite mais coordenadas: ");
    }

    printf("\nEncerrado.\n");
    return 0;
}

```

(DEEPSEEK, 2026)

Em Prolog, podemos reproduzir o mesmo esquema a partir de predicados de forma muito mais simples, veja o exemplo:

```

% observe com atenção, os argumentos dos predicados "ponto"
vertical(linha(ponto(X,Y1),ponto(X,Y2))).
horizontal(linha(ponto(X1,Y),ponto(X2,Y))).

% a partir disto, já podemos realizar as consultas:
?- vertical(linha(ponto(1,1),ponto(1,3))).
true.

?- vertical(linha(ponto(1,1),ponto(2,3))).
false.

?- horizontal(linha(ponto(1,1),ponto(3,1))).
true.

?- horizontal(linha(ponto(1,1),ponto(3,2))).
false.

```

O que acabamos de fazer é chamado de unificação, porém damos termos já conhecidos para o predicado *linha*, isso é “simples” em Prolog podemos ser mais gerais e perguntar

```
% repare bem nas variáveis!
```

```
?- vertical(linha(ponto(1,1),ponto(X,Y))).
```

```
X = 1.
```

```
?- vertical(linha(ponto(1,1),ponto(1,Y))).
```

```
true.
```

```
% porque _?
```

```
?- vertical(linha(ponto(1,1),ponto(X,_))).
```

```
X = 1.
```

```
?- vertical(linha(ponto(1,1),ponto(2,Y))).
```

```
false.
```

```
?- vertical(linha(ponto(1,1),P)).
```

```
P = ponto(1, _).
```

```
?- vertical(linha(ponto(1,_),P)).
```

```
P = ponto(1, _).
```

```
% podemos até mesmo ter a própria definição de vertical!
```

```
?- vertical(linha(P1,P2)).
```

```
P1 = ponto(_A, _),
```

```
P2 = ponto(_A, _).
```

4.7 Backtracking

```
mulher(marge).
mulher(lisa).
mulher(maggie).

homem(homer).
homem(bart).

pais(marge, bart).
pais(homer, bart).

pais(marge, lisa).
pais(homer, lisa).

pais(marge, maggie).
pais(homer, maggie).

pai(X, Y) :-
    pais(X, Y),
    homem(X).

?- pai(X, bart).
% qual será a resposta?
```

Na regra para encontrar o pai, o Prolog passa pelos predicados procurando pelo predicado *pai*, e então, em Y é instanciado “bart”, agora Prolog procura por um parente X que tenha como filho, “bart”. O primeiro fato encontrado será `pais(marge, bart)`, agora X é “marge”, no próximo predicado, Prolog checa se X é homem, por X não ser homem, ele realiza o que chamamos de “rastreamento para trás”, ou *backtracking*; que é quando Prolog irá continuar a busca do último ponto que teve sucesso. Dessa vez, encontrando homer, checa se homer é um homem, e retorna X como “homer”.

Recursão

% ref: <https://www.youtube.com/watch?v=PR-Ab0nv6J4>

```
fatorial(1, 1) :- !.
fatorial(N, Nfatorial) :-
    M is N - 1,
    fatorial(M, Mfatorial),
    Nfatorial is Mfatorial * N.
```

Tabela 1 – Análise SWOT

Pontos fortes: • Sintáxe e semântica simples • Estruturas de dados persistentes e imutáveis, com ponteiros “declarativos” • Precisão aritmética arbitrária • Segurança (garbage collection, exceções Null-Pointer inexistentes, ...) • Inferência eficiente, casamento de padrões e unificação • Indexação e tabelamento eficiente • Desenvolvimento e debugging rápido com REPL (loop de leitura, execução, impressão) • Aplicações de sucesso: – Análise de programas – Processamento de linguagem natural – Sistemas especialistas, PAT (Prova automática de teoremas)	Pontos fracos: • Sintaticamente diferente das LPs “tradicionais”, não é uma linguagem <i>mainstream</i> • Curva de aprendizado íngreme; iniciantes podem facilmente escrever programas que entram em loop ou consomem muitos recursos computacionais • Tipagem estática • Portabilidade limitada • Bibliotecas/pacotes: avaliabilidade e gerência • IDEs e ferramentas de desenvolvimento possuem capacidades limitadas em algumas áreas (como a refatoração) • Desenvolvimento de interfaces de usuário
Oportunidades: • Novas áreas de aplicação – IA neurosimbólica – IA explicável e verificável – <i>Big Data</i> • Novas funcionalidades e desenvolvimentos – Raciocínio probabilístico – Paralelismo (como é feito em Lean) – Compilador JIT – Integrando resolução de ASPs (Programação de conjunto de resposta) e SATs (Problema de satisfatibilidade booliana) ou SMTs (Teorias de satisfatibilidade módulo).	Ameaças • <i>User base</i> baixa, quando comparada com outras LPs • Comunidade fragmentada • Parte ativa da comunidade com novas implementações e funcionalidades constantemente desenvolvidas criam fragmentação • Novas linguagens de programação • A percepção de ser uma LP “antiga” • Imagem errada por ensino “raso” da linguagem • O mundo pós-desktop das aplicações web em JavaScript

Fonte: (KÖRNER et al., 2022), Adaptado

5 Linguagens Relacionadas

- PLANNER é uma linguagem influenciadora
- Bend (linguagem pt-br) é provavelmente uma linguagem de alguma forma relacionada, apesar de talvez não ter noção disso; em particular, Bend 2
- Linguagens OOP notórias como Java e C++ parecem ser opostas

6 Exemplo de Programas em Prolog

7 Considerações Finais

Referências

COLMERAUER, A.; ROUSSEL, P. The birth of prolog. *SIGPLAN Not.*, Association for Computing Machinery (ACM), v. 28, n. 3, p. 37–52, mar. 1993.

DEEPSEEK. *DeepSeek Flash*. Zhejiang, China: [s.n.], 2026. Inteligência Artificial. Prompt 1: Replique em C, utilizando structs como definir e consultar o seguinte algoritmo em Prolog (não é necessário que em C responda como em prolog, mas que chegue no mesmo resultado: `vertical(linha(ponto(X,Y),ponto(X,Z))). horizontal(linha(ponto(X,Y),ponto(Z,Y)))`). Prompt 2: modifique para que ao invés de um mock com as perguntas, que seja um scanf perguntando os números separados por espaço, em while loop. Disponível em: <https://deepseek.com/en/>.

FISHER, L. M. *In Memoriam Alain Colmerauer: 1941-2017*. 2017. Website. Disponível em: <https://cacm.acm.org/news/in-memoriam-alain-colmerauer-1941-2017/>.

KÖRNER, P. et al. *Fifty Years of Prolog and Beyond*. 2022. Disponível em: <https://arxiv.org/abs/2201.10816>.

RESEARCH, I. *Watson and the Jeopardy! Challenge*. 2013. Website. Disponível em: <https://www.youtube.com/watch?v=P18EdAKuC1U>.